

Exercício 1

Parte A: O livro AIMA divide as definições de Inteligência Artificial em quatro categorias: o foco no processo de raciocínio (**pensar**) contra o comportamento (**agir**), e o foco na fidelidade à performance humana (**humano**) contra a otimização de resultados (**racional**). A abordagem adotada por Russell e Norvig concentra-se, estritamente, em agir racionalmente, de modo que, um agente racional (inteligente) é aquele que age para alcançar o melhor resultado possível ou, quando há incerteza, o melhor resultado esperado. O foco não está em replicar o comportamento e pensamento humano, nem em apenas deduzir pensamentos logicamente corretos, mas também em tomar as ações autônomas que maximizam o sucesso no ambiente.

Parte B: Sob a ótica da abordagem de agir racionalmente, a inteligência de um agente não é aferida pela mimetização do raciocínio humano (como proposto no Teste de Turing), mas sim de forma pragmática e objetiva por meio de seu comportamento no ambiente. Para que essa avaliação ocorra, estabelece-se uma medida de desempenho: um critério externo que quantifica o grau de sucesso do agente. Nesse contexto, a inteligência é definida estritamente pela capacidade de escolha: um agente é considerado racional se, e somente se, a ação selecionada é aquela que maximiza a expectativa da medida de desempenho, baseando-se na sequência de percepções acumulada e nos conhecimentos intrínsecos disponíveis no momento da decisão.

Exercício 2

O Capítulo 2 do livro AIMA categoriza os programas de agentes em cinco tipos principais, que aumentam em grau de complexidade e capacidade de lidar com diferentes ambientes:

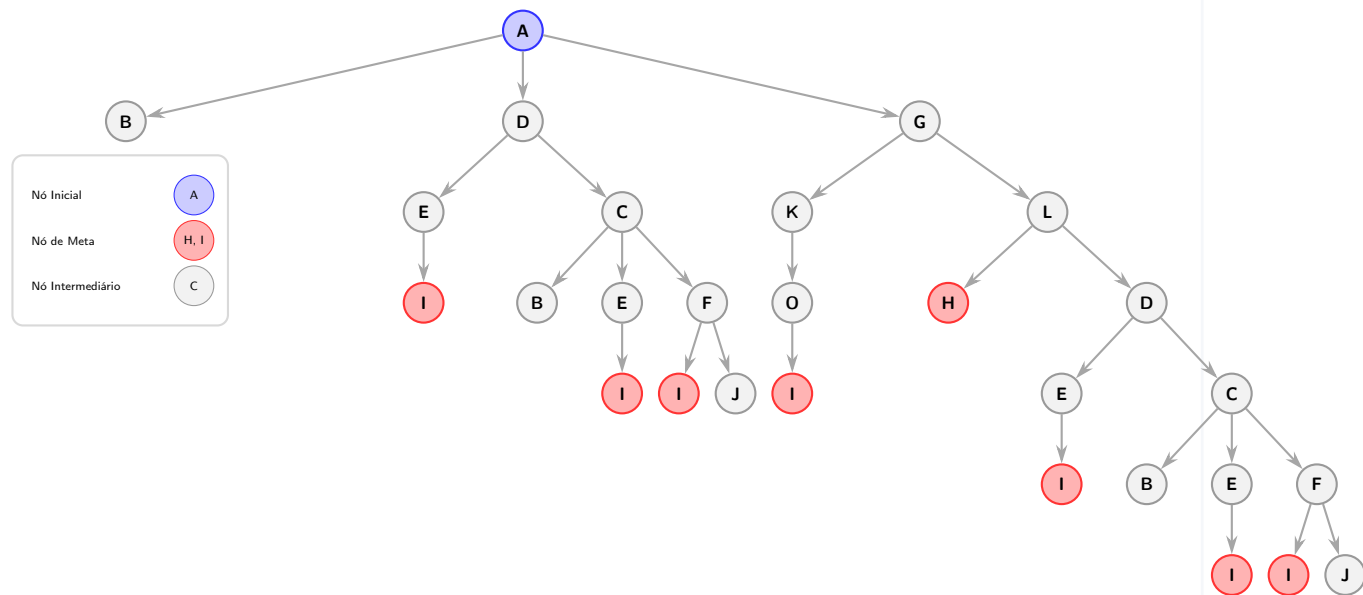
- 1. Agentes Reativos Simples (Simple Reflex Agents):** Estes agentes selecionam ações com base *apenas* na percepção atual do ambiente, ignorando todo o histórico de percepções passadas. Eles operam por meio de regras de condição-ação (ex: "se o carro da frente frear, então freie"). Só funcionam corretamente se o ambiente for totalmente observável.
- 2. Agentes Reativos Baseados em Modelo (Model-based Reflex Agents):** Ao contrário dos agentes simples, estes conseguem lidar com ambientes parcialmente observáveis. Para isso, mantêm um **estado interno** que rastreia a parte do mundo que não pode ser vista no momento. Esse estado é atualizado com base no histórico de percepções e em um "modelo" de como o mundo evolui de forma independente e como as ações do agente afetam esse mundo.
- 3. Agentes Baseados em Objetivos (Goal-based Agents):** Saber o estado atual não é suficiente; o agente precisa de **objetivos** que descrevam situações desejáveis. Diferente dos agentes reativos que agem por instinto (regras fixas), estes consideram o futuro ("O que acontecerá se eu fizer isso?") para escolher a ação que o aproximará do seu objetivo. Isso frequentemente envolve algoritmos de busca e planejamento.
- 4. Agentes Baseados em Utilidade (Utility-based Agents):** Objetivos fornecem uma visão binária (alcançou ou não alcançou). Os agentes baseados em utilidade vão

além, utilizando uma **função de utilidade** que mapeia um estado (ou sequência de estados) para um número real, indicando o grau de "felicidade" ou eficiência daquele estado. Eles procuram maximizar a utilidade esperada, sendo úteis quando há múltiplos caminhos para um objetivo ou objetivos conflitantes.

5. **Agentes com Aprendizado (Learning Agents):** Qualquer um dos agentes anteriores pode ser transformado em um agente com aprendizado. Eles nascem com um conhecimento inicial básico, mas possuem componentes (como o elemento de aprendizado e o elemento de crítica) que lhes permitem **aprender com a experiência**. Isso permite que operem em ambientes inicialmente desconhecidos e se tornem mais competentes ao longo do tempo, ajustando seu comportamento com base no feedback recebido (recompensas ou penalidades).

Exercício 4

- A
 - 1. **Busca Não-Informada:** O algoritmo não tem conhecimento sobre o quão longe está do objetivo, apenas gera sucessores e distingue se um estado é objetivo ou não.
 - 2. **Busca Informada:** Utiliza conhecimento específico do domínio através de uma função heurística $h(n)$, que estima o custo do estado atual até o objetivo.
- B
 - 1. **Busca em Árvore:** O algoritmo não mantém um registro de estados já visitados. Se o espaço de estados contiver ciclos ou múltiplos caminhos para o mesmo nó, a busca em árvore pode entrar em loop infinito.
 - 2. **Busca em Grafo:** Inclui uma estrutura de dados para armazenar todos os estados já visitados. Isso evita a exploração redundante de estados e evita ciclos infinitos.
- C
 - 1. **Busca por Planejamento de Ações:** O foco está na sequência de passos necessária para chegar ao objetivo. O caminho é a solução
 - 2. **Busca por Identificação de Estados:** O foco está apenas no estado final que satisfaz as condições do problema. O caminho percorrido é irrelevante
- D
 - 1. **Busca Local:** Opera sobre um único estado atual e tenta melhorá-lo movendo-se para vizinhos. Não mantém um caminho e é ideal para otimização.
 - 2. **Busca CSP:** Busca em espaços onde os estados são definidos por variáveis que devem satisfazer um conjunto de restrições.
- E
 - 1. **Busca Offline:** O agente calcula a solução completa antes de executar a primeira ação. Assume um ambiente estático e conhecido
 - 2. **Busca Online:** O agente intercala computação e ação. Ele expande um nó, executa uma ação, observa o novo estado e então decide o novo passo.
- F
 - 1. **Busca Completa:** Um algoritmo é considerado completo se ele garante encontrar uma solução, caso exista, em tempo finito
 - 2. **Busca Ótima:** Um algoritmo é ótimo se ele garante encontrar a melhor solução entre todas as possíveis

Exercício 6

4.1 : A -> B -> D -> G -> C -> E -> L -> K -> B -> F -> I

4.2 : A -> B -> D -> G -> C -> E -> L -> K -> F -> I

4.3 : A -> B -> D -> C -> B -> E -> I

4.4 : A -> B -> D -> C -> E -> I

4.5 : Lim 0: A

Lim 1: A -> B -> D -> G

Lim 2: A -> B -> D -> C -> E -> G -> L -> K

Lim 3: A -> B -> D -> C -> E -> F -> E -> I (META)

Lista Final: A -> A -> B -> D -> G -> A -> B -> D -> C -> E -> G -> L -> K -> A -> B -> D -> C -> E -> F -> E -> I

Exercício 10

1. Verdadeiro.
2. Falso, como a busca gulosa ignora o custo do caminho e toma as decisões baseadas exclusivamente no menor valor de $h(n)$ se vemos o vizinhos de A disponíveis que são B ($h = 20.0$), C ($h = 33.54$) e K ($h = 40.0$) o algoritmo greedy inevitavelmente escolherá o B
3. Verdadeiro
4. Falso, para que algoritmos de busca heurística garantam otimalidade, a heurística precisa ser admissível, ou seja, por definição ela deve sempre subestimar ou ser igual ao custo real
5. Falso, heurísticas admissíveis garantem a completude e otimalidade do A* apenas quando a implementação utiliza a busca em árvore. Se a implementação utilizar a busca em grafo, uma heurística pode fechar um estado por um caminho subótimo antes de encontrar o caminho ótimo
6. Verdadeiro
7. Verdadeiro
8. Falso, a relação de implicação é no sentido inverso. Toda heurística consistente é obrigatoriamente admissível, no entanto, é possível construir uma heurística que seja admissível mas que seja inconsistente.

Exercício 13

- a. $h_1 + h_2$: não é admissível. Contra-exemplo: Suponha um estado s onde o custo até a meta seja $h^*(s) = 10$. Sejam as estimativas $h_1(s) = 8$ e $h_2(s) = 7$, a heurística induzida resultaria em $h(s) = 15$ que não é admissível
- b. $h_1 \times h_2$: não é admissível. Contra-exemplo: Suponha um estado s onde o custo até a meta seja $h^*(s) = 10$. Sejam as estimativas $h_1(s) = 8$ e $h_2(s) = 7$, a heurística induzida resultaria em $h(s) = 56$ que não é admissível
- c. $\max\{h_1, h_2\}$: é admissível, como sabemos por premissa que tanto $h_1(s) \leq h^*(s)$ e $h_2(s) \leq h^*(s)$ é logicamente impossível que o valor máximo entre eles exceda $h^*(s)$
- d. $1 + \min\{h_1, h_2\}$: não é admissível. Contra-exemplo: Suponhamos $h^*(s) = 0$, como h_1 e h_2 devem ser admissíveis para todos os estados, temos que $h_1(s) = h_2(s) = 0$, avaliando a heurística. $h(s) = 1 + \min\{0, 0\} = 1$, como $1 > 0$, não é admissível
- e. $\alpha h_1 + \beta h_2$, com $\alpha \geq 0, \beta \geq 0$ e $\alpha + \beta = 1$. Partindo das premissas de admissibilidade e multiplicando cada heurística por α e β (que são não negativos, logo não invertem o sinal da inequação):

$$\alpha h_1(s) \leq \alpha h^* \quad (1)$$

$$\beta h_2(s) \leq \beta h^* \quad (2)$$

$$\alpha h_1(s) + \beta h_2(s) \leq \alpha h^* + \beta h^* \quad (3)$$

$$\alpha h_1(s) + \beta h_2(s) \leq (\alpha + \beta) h^* \quad (4)$$

$$\alpha h_1(s) + \beta h_2(s) \leq (1) h^* \quad (5)$$

Logo, a heurística é provada como admissível